Team: Keyboard Gremlins

COS301 Capstone Project
University of Pretoria

Hackathon Platform

SAS

Name	Student Number
Ayush Sanjith	23535424
Jasveer Ramdhaw	23537036
Rene Reddy	23558572
Heinrich Römer	23538181
Kwaku Asiedu (Sam)	18100156

Team Contact: keyboardgremlins@gmail.com

1. Introduction

The Hackathon Platform is a cloud-agnostic web application purpose-built for running deterministic optimisation hackathons. Organisers (Admins) configure events, upload levels and an official grading solver, and operate the event live. Participants register, form teams, download problem material, and submit a solution output together with a source-code archive for each level. Submissions are graded asynchronously by the official solver, and results are surfaced to participants through submission history, logs, and a live leaderboard.

1.1 Purpose and Scope

- The architectural and design patterns used across the frontend and backend, and why they were chosen.
- The constraints - technical, organisational, and academic - that shaped those decisions.
- A technology-neutral architectural diagram of the system's logical structure.
- A mapping from each non-functional (quality) requirement in the SRS to the concrete architectural mechanism that satisfies it.
- The technology stack and the justification for each choice.
- How the system is actually deployed, including a technology-specific deployment diagram and CI/CD pipeline.

2. Architectural Requirements

The Hackathon Platform follows a Client-Server architecture. The Angular single-page application (the client) never talks to the database or object storage directly - every interaction goes through the Spring Boot REST API (the server), which is the sole owner of business rules, persistence, and file access. Within that client-server split, each side is organised by its own

internal pattern, and the asynchronous scoring path is organised as a separate event-driven subsystem.

2.1 Architectural Patterns

Client-Server

The platform has two user populations with sharply different needs (Admins configuring and operating an event vs. Participants competing in one) but a single, non-negotiable trust boundary - the official solver and every stored file must be controlled entirely by the server. A fat client or a server-rendered monolith would blur that boundary by letting client-side code influence what gets graded or how files are accessed. Splitting into a thin client and an authoritative server keeps every trust-sensitive decision in one place, and lets the Admin and Participant portals be built as two thin views over the same backend rather than two separately-trusted systems.

Therefore, the system is a two-tier client-server application. The Angular client is a static bundle served independently of the API and communicates exclusively over HTTPS using JSON REST calls (plus one long-lived Server-Sent Events connection for live leaderboard updates). This keeps the client stateless with respect to business logic and lets the frontend and backend be built, tested, scaled, and deployed independently - which matters for a two-portal product (Admin portal and Participant portal) sharing one backend.

MVVM on the frontend (Angular)

MVVM was chosen specifically because it lets the ViewModel be unit tested within complete isolation from the DOM (no component render required), while the View stays thin enough that a UI restyle never risks touching business logic.

The Participant and Admin portals follow a Model-View-ViewModel pattern, which is the natural shape of an Angular application:

- View - Angular components and their HTML templates. Views are deliberately "dumb": they bind to observable state and re-render when it changes, but contain no business logic.

- ViewModel - Angular injectable services. Each service wraps HttpClient calls to the backend, exposes RxJS Observables/Signals as bindable state, and mediates between the View and the Model.
- Model - TypeScript interfaces/DTOs that mirror the backend's request/response contracts, plus route guards (auth.guard.ts) and an HTTP interceptor (auth.interceptor.ts) that attaches the JWT to outgoing requests and centralises 401/403 handling.

Two-way data binding and the async pipe keep the View reactive to ViewModel state changes (e.g. a new leaderboard entry pushed over SSE flows straight through the leaderboard service into the participant's view without manual DOM manipulation).

MVC on the backend (Spring Boot)

Two important properties the SRS weighs heavily are testability (≥80% coverage, JUnit-driven CI gate) and auditability of access control. Both push toward the same structural decision, keep HTTP parsing, business rules, and persistence in three separate layers rather than letting one class do all three. A controller that both validates a solver upload and decides how it is stored cannot be unit-tested without spinning up HTTP; a service that also runs SQL cannot be tested without a live database. Separating them means the business rule "an Admin may only rescore submissions for events they own" is expressed once, in a service method, and is exercised by a fast JUnit test with a mocked repository.

- Controller - REST controllers that own request/response mapping, input validation, and route security, and delegate everything else to a service.
- Model - JPA entities that represent persistent domain state and are mapped to the relational schema via Spring Data JPA.
- Controller -> Service -> Repository - rather than controllers manipulating entities directly, a Service layer holds all business rules, and a Repository layer (Spring Data JPA interfaces) is the only thing that talks to Postgres. This keeps controllers thin and business logic unit-testable independent of HTTP.

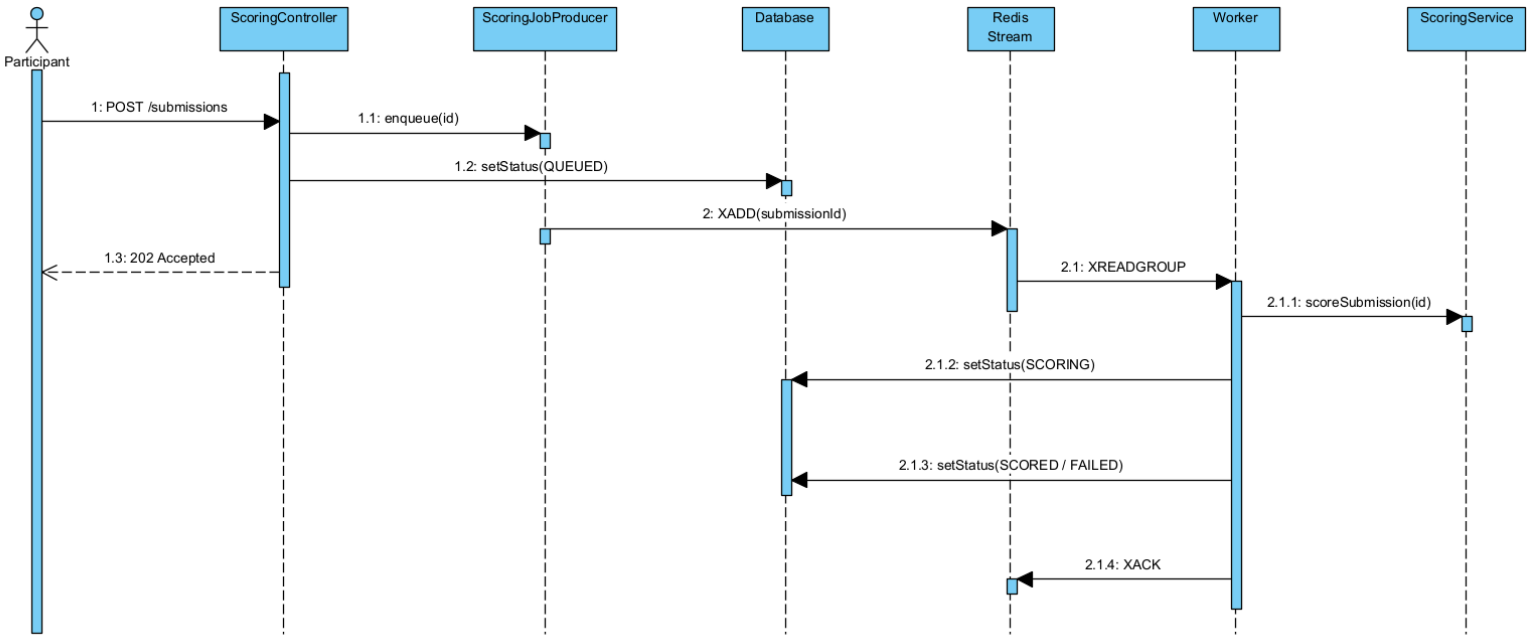
Data Transfer Objects are used at the controller boundary instead of exposing JPA entities directly, so that the public API contract can evolve independently of the database schema.

Scoring Orchestration

Purpose

Scoring a submission means running an external Python solver as a subprocess. This process can take a few minutes to run depending on the solver complexity. By doing this synchronously inside the upload request:

- Keeps an HTTP thread open for the duration of solver execution and risks holding other server resources if scoring is done synchronously
- Has no upper bound on how many solver subprocesses run at once, where a sudden increase of submissions can exhaust hardware
- Does not allow scaling of the scoring capacity independently of the API
- The queue allows decoupling between submission upload and submission scoring.



Components

Class	Responsibility
ScoringJobProducer	Sets the submission to “QUEUED” and pushes it into the Redis stream.
ScoringJobConsumer	Reads one job from the Redis stream and calls the scoreSubmission method which acknowledges the stream on completion.
ScoringJobReclaimer	This is a scheduled task that periodically claims jobs that were delivered but never acknowledged post idle threshold (1 min) and reprocesses them.
Scoring Service	This contains the scoring logic, where the database transactions are split so that the solver subprocess executes outside an active transaction.

Advantages of a consumer group (Redis Streams):

- Multiple app instances can run workers against the same stream without two workers scoring the same submission.
- Pending Entries List per consumer: if a worker crashes, the entry remains in the list, which is what the JobReclaimer depends on.
- Horizontal Scaling: Adding workers or app instances increases the number of consumers processing the shared stream.

Bounded Concurrency

ScoringQueueConfig creates the stated amount of workers received from the input. Each worker is processed by its own thread from a fixed size pool. This amount caps the amount of solver subprocesses that can run in one app instance. This is the mechanism that prevents an upload burst from spawning unbounded parallel Python processes.

Scalability

Scaling within one instance

Throughput is bounded by the specified number of solver subprocesses that are allowed to run at once. Raising this amount will increase the throughput until the host's hardware limits are reached. Increasing the number of concurrent workers after this point actually creates more contention rather than throughput. The current implementation does not include dynamic tuning, this is a fixed value that is chosen when the instance is created.

Horizontal Scaling

The current implementation is horizontally scalable since workers join a shared redis consumer group. Even running multiple instances of the application, Redis streams distributes each stream entry into a single consumer within the consumer group. If a consumer fails before acknowledging the entry, this pending entry can be reclaimed by another consumer.

Handling submission spikes

The Redis stream absorbs the burst and workers process queued submissions at the rate of the specified concurrency value. Participants see the status of their submission is changed to "QUEUED" immediately, there is no risk of uploads timing out or piling up on Tomcat, which was the core motivation for creating an asynchronous design.

Reliability

Redis Streams and consumer groups include an at least once message delivery. This ensures that a submission is only acknowledged after scoring is successful, therefore if an error occurs before scoring the submission remains in the Pending Entries List. The implemented reclaimer periodically claims stale pending entries and reprocesses them to ensure that these submissions do not get lost.

2.2 Design Patterns

Pattern	Where used	Purpose
Facade	FileMetadataService, sitting in front of LevelFileRepository, SolverVersionRepository, SubmissionRepository, and HackathonRepository	Callers that just want "the metadata for this file" shouldn't need to know whether the file in question is a level resource, a solver version, or a submission output - each backed by a different repository/entity. FileMetadataService gives one simplified entry point over all four, which is what let the Storage and Scoring controllers stay thin instead of each re-implementing "which repository does this file belong to."
Strategy and Adapter	StorageService interface with AzureBlobStorageService implementation	Business and controller code programs against StorageService only, so the object-storage provider can be swapped without touching a single line of controller or service logic.
DTO (Data Transfer Object)	dto package - EventRequest, LoginRequest, TeamResponse, SubmissionResponse, LeaderboardEntryResponse, ScoringLogResponse, AuthResponse, etc.	Decouples the wire-level API contract from the JPA entity graph, preventing accidental exposure of internal fields and letting the schema evolve independently of the API.
Producer/Consumer (Observer-style)	ScoringJobProducer / ScoringJobConsumer over a Redis Stream consumer group	Decouples submission intake from grading; multiple consumers can subscribe and scale horizontally without the producer knowing how many are listening.
Chain of Responsibility	JwtAuthFilter within the Spring Security filter chain	Chain of Responsibility lets JwtAuthFilter validate the token and populate the security context, then hand off to the next filter (or short-circuit with 401) without any filter needing to know what the others do.

Factory Method	@Bean-annotated factory methods inside @Configuration classes - blobServiceClient() in AzureBlobConfig, and the Redis stream/consumer-group setup in ScoringQueueConfig	Object construction that depends on externalised configuration (connection strings, container names, stream/group names from application.yml) is isolated in one factory method per resource, so the rest of the codebase injects a ready-to-use client without knowing how it was assembled or from which environment's config values.

2.3 Constraints

Technical constraints

- Open-source, cloud-agnostic technology stack: the platform must be deployable to any container-capable infrastructure without vendor lock-in on compute (open-source database, backend framework, and frontend framework).
- The official solver is an untrusted, event-organiser-supplied Python script; it must run with a bounded timeout, in an isolated working directory, with output capped in size, so a malformed or malicious solver cannot exhaust platform resources or block the scoring pipeline.
- The platform must never execute participant-submitted code - only the organiser-supplied solver executes; participant source archives are stored for audit purposes only.
- Stateless authentication: horizontal scaling of the API requires that no session state be held in server memory, so JWTs and Spring Security's stateless session policy are mandatory.

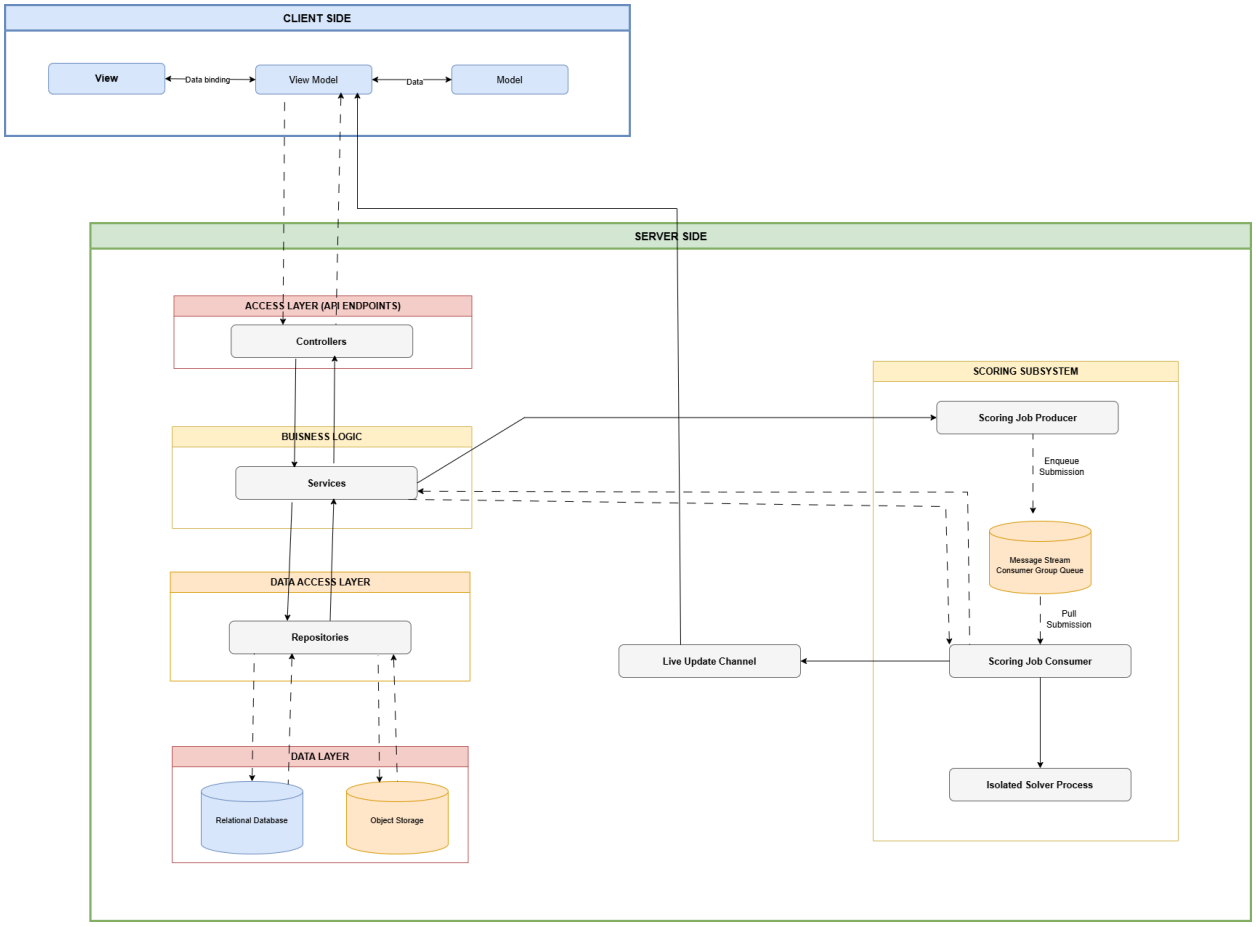
Business constraints

- The system must be reachable via a public URL for evaluation, with a reproducible deployment.
- Multi-tenancy (multiple organisations running isolated events from one deployment) is an optional, not core, requirement, and the current schema is designed so tenant isolation can be layered on later without a rewrite.
- Budget constraints: Base/Free tiers were used for deployment/infrastructure which impacts the load capability and response time of the system.

- Policy constraints: Due to azure subscription provided by the university, policy constraints limit our server location which impacts response times.

2.4 Architectural Diagram

The diagram below is technology-neutral: it shows the logical structure of the system.



2.5 Mapping Quality Requirements to Architectural Decisions

The five quantified quality requirements carried over from the SRS are mapped below to the concrete mechanism in the current architecture that satisfies each one.

Quality Requirement	Architectural Decision
Scalability: System can handle 100 concurrent users	The application is deployed as containerized services on Azure Container Apps with horizontal scaling support. Stateless application design enables handling increased concurrent traffic.
Reliability: Less than 0.1% of failed HTTP requests during load testing	Centralized exception handling, input validation and health checks and monitoring Azure Logs to detect and recover from failures.
Performance: 95% of requests complete within 10.5 seconds with 100 concurrent users	Optimized database queries, minimizing API calls and processing long running operations asynchronously.
Availability: 99.95% uptime excluding scheduled maintenance	Host the platform on Azure using Azure Container Apps, using Azure's 99.95% SLA.
Maintainability: Test coverage of 80% or higher	Using CI/CD to automatically run unit, integration and end to end tests. SonarCloud enforces code quality and CodeCov monitors and reports testing coverage.

2.6 NFR Traceability Matrix

Quantified Requirement	Tactic in SAS	Test/tool	Target / actual
95% of requests complete within 30sec at 100 concurrent users	Async scoring with Redis Streams Queue keeps HTTP threads unblocked and a bounded worker pool prevents subprocess contention.	k6	<30s / 25s
Error rate stays below 20% at 100 concurrent users	Centralized exception handling	k6	<20% / 18%
99.95% uptime, excluding scheduled maintenance	Azure Container Apps SLA	UptimeRobot [https://stats.uptimerobot.com/edALMds6Gt/803607544]	>99.95% / 100%
Test coverage >55%	Controllers, Services and Repositories are tested using unit and integration tests.	CodeCov	>55% / 59%
Lighthouse performance	Angular production build	Lighthouse CI	Threshold of 90

3. Technology Requirements

The technology stack was chosen to support the core requirements identified above: configurable event management, an asynchronous scoring pipeline capable of absorbing submission spikes, near-real-time leaderboard updates, and cloud-agnostic, containerised deployment.

Layer	Technology	Reason
Frontend	Angular, TypeScript 5, RxJS	Component architecture suits the dual Admin/Participant portal model; RxJS gives the reactive ViewModel layer.
Backend API	Java 21, Spring Boot	Production grade, strongly typed REST framework with mature security, validation, and ORM support.
Authentication	JWT + Spring Security + BCrypt	Stateless auth suited to horizontal scaling; RBAC enforced via method security.
Relational database	PostgreSQL, versioned with Flyway migrations	Open-source, ACID-compliant store for events, teams, submissions, scores, and audit data; Flyway keeps schema changes reproducible across environments.
Object storage	Azure Blob Storage	Stores problem specs, level files, solver versions, participant outputs, source archives, and branding assets.
Async job processing	Redis 7 (Streams + consumer groups)	Backs the scoring job queue. Also reserved for caching as the platform grows.
Real-time updates	Server-Sent Events over HTTPS	This supersedes the WebSocket layer described in the tender proposal. SSE was adopted instead because leaderboard updates are one-directional (server -> client), which SSE serves with a simpler connection model, automatic reconnect, and easier compatibility

		with standard HTTP infrastructure than a bidirectional WebSocket.
Containerisation	Docker and Docker Compose for local development	Consistent, portable environments across dev, CI, and production cloud-agnostic by design.
CI/CD & quality	GitHub Actions, Spotless, ESLint, JUnit, Jasmine/Karma, Playwright, SonarCloud, Codecov, Lighthouse CI, k6	Automated build, lint, test, and quality gates on every push/PR.

Deployment

Live System

The frontend is available at https://hackathonplatform.co.za and the backend (API) is available at https://api.hackathonplatform.co.za. The live system is deployed from the main branch, where CD automatically deploys the branch on push.

To reproduce the deployment, run the following commands:

Git clone https://github.com/COS301-SE-2026/Hackathon-Platform

Cd Hackathon-Platform

Docker build -f docker/backend.Dockerfile -t hackathon-backend ./backend

Docker build -f docker/frontend.Dockerfile -t hackathon-frontend ./frontend

Environment Parity

	Development	Production
Backend	mvn spring-boot:run	Azure Container App
Frontend	ng serve	Azure Container App

Database	Postgres container	Azure PostgreSQL Flexible server
Cache	Redis container	Azure Managed Redis
Deploy trigger	Manual	Push to main

Secrets Management

- Github Actions secrets contain all secrets needed for running the CI and CD.
- Container App environment variables contain all secrets needed for running both frontend and backend.

Rollback Strategy

How images are tagged

Every push to main builds and pushes 2 images to the Azure Container Registry:

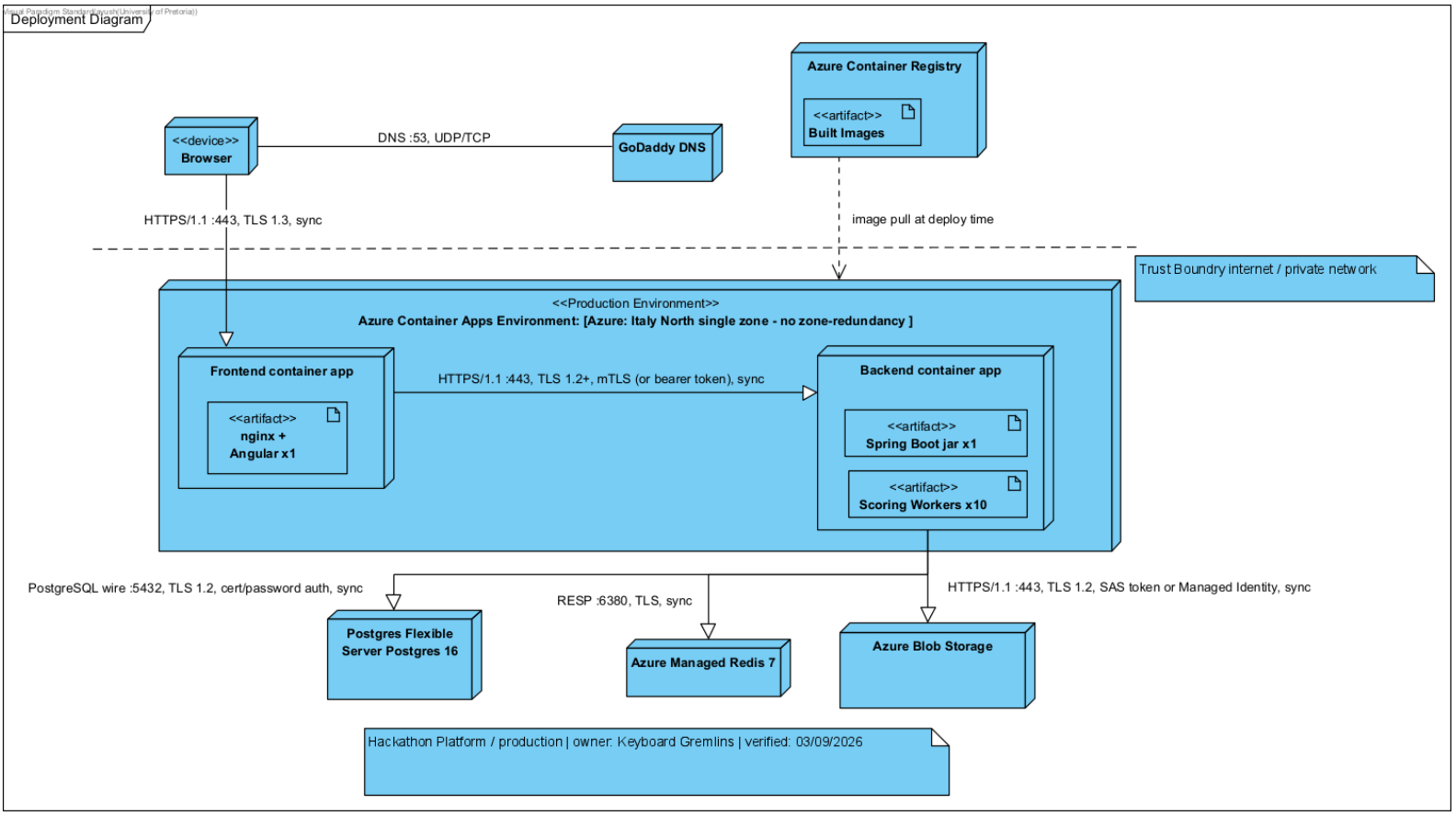
- backend:<timestamp>-<short-sha> and frontend:<timestamp>-<short-sha>: Are unique tags for builds.
- backend:latest and frontend:latest always point to the most recent build.

Strategy: re-deploy-previous-tag

Steps:

1. Find the CD in the Actions tab in Github.
2. Find the last run before the failed one that succeeded.
3. Open it and note the image tag it used.
4. Re-deploy using that tag with workflow_dispatch.

Deployment Diagram


Deployment Diagram
Azure Container Registry
<<artifact>>
Built Images
<<device>>
Browser
DNS :53, UDP/TCP
GoDaddy DNS
HTTPS/1.1 :443, TLS 1.3, sync
image pull at deploy time
Trust Boundry internet / private network
<<Production Environment>>
Azure Container Apps Environment: [Azure: Italy North single zone - no zone-redundancy]
Frontend container app
<<artifact>>
nginx +
Angular x1
HTTPS/1.1 :443, TLS 1.2+, mTLS (or bearer token), sync
Backend container app
<<artifact>>
Spring Boot jar x1
<<artifact>>
Scoring Workers x10
PostgreSQL wire :5432, TLS 1.2, cert/password auth, sync
RESP :6380, TLS, sync
HTTPS/1.1 :443, TLS 1.2, SAS token or Managed Identity, sync
Postgres Flexible
Server Postgres 16
Azure Managed Redis 7
Azure Blob Storage
Hackathon Platform / production | owner: Keyboard Gremlins | verified: 03/09/2026

CI/CD Pipeline Flowchart